

```
In [336... import datetime as dt
import pandas as pd
import numpy as np
import warnings
#import statsmodels.api as sm
import matplotlib.pyplot as plt
from scipy.stats import zscore
from sklearn.metrics import mean_squared_error, r2_score

warnings.filterwarnings('ignore')
```

```
In [337... def isclose(value, original, tolerance= 0.05):
    return value <= original * (1+tolerance) and value >= original * (1-tolerance)
```

Abstract

For this project, I chose the option of making my own trading algorithm. The algorithm in my project relies on Fourier transform to extract patterns of repetition in stock market data. The price of an asset can fluctuate around the fair price for the following reason: when buyers realize the actual price is overvalued compared to its fair, they stop buying and start selling, driving down the price; inversely, when buyers realize the actual price is undervalued compared to its fair, they stop selling and start buying, driving up the price. This behavior would translate into a harmonic motion of sorts around the fair price, and thus the algorithm can take advantage of this phenomenon.

Methods

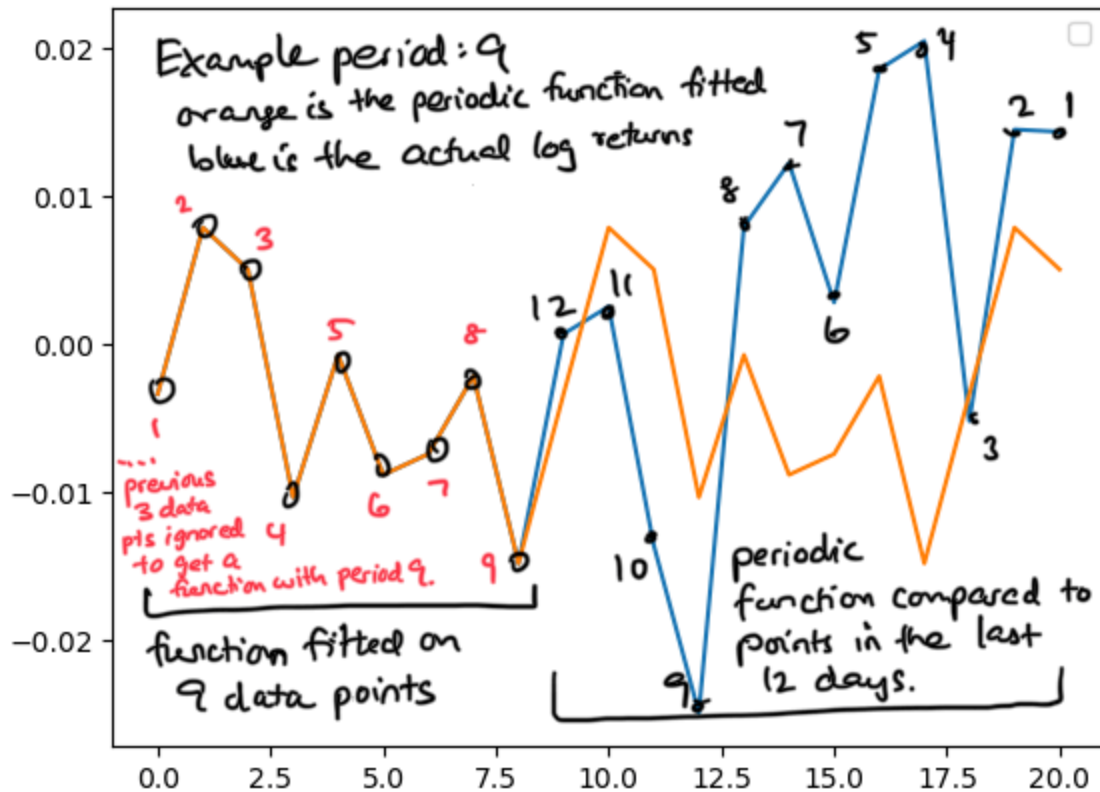
The dataset used for this project was the stock_data.parquet file provided for Lab 02. The algorithm is as follows:

Let's say we have 24 days of data on a single asset's log return behind us.

1. Using the log returns from the last 24 days, we can try to extract the effective frequency that the log returns of the asset fluctuates at. To do this, we fit data points from 24 days ago until to 12 days ago with multiple periodic functions, one with period 7, one with period 8, ..., and one with a period of 12 days.
2. For each of these functions:
 - A. We can decide to filter out frequencies that aren't as strong as others.
 - B. We need to measure how effectively it can predict the log returns within the last 12 days. To do this, the function is extended to cover the last 12 days, and the MSE metric is used to measure how closely the periodic function can predict the last 12 days.

3. We select the periodic function with the least MSE. Note that the frequency of this function is the effective frequency of oscillation within the data locally (in the last 12 days).
4. With the chosen periodic function, we can then try to predict the forward log return (the log return of the next day) by extending the periodic function into the next day. The position we have for the current day will be proportional to this predicted forward log return.

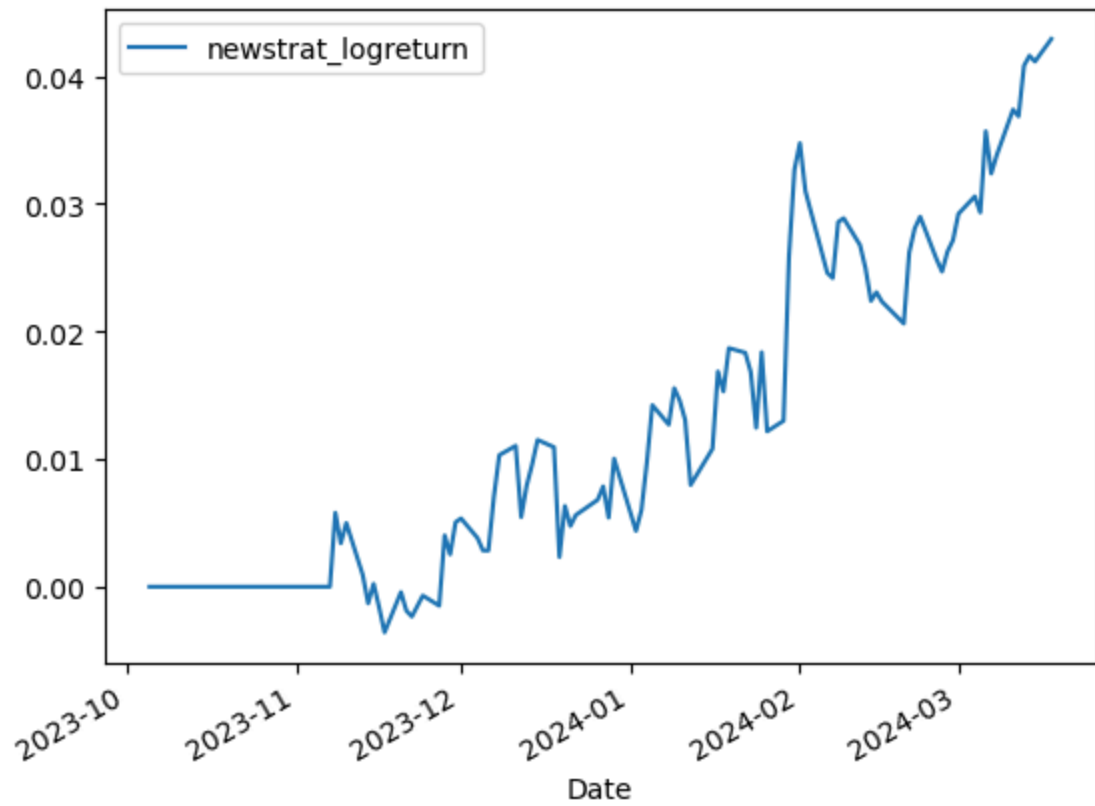
See the image below for an example of what goes on in Steps 2.1 and 2.2.



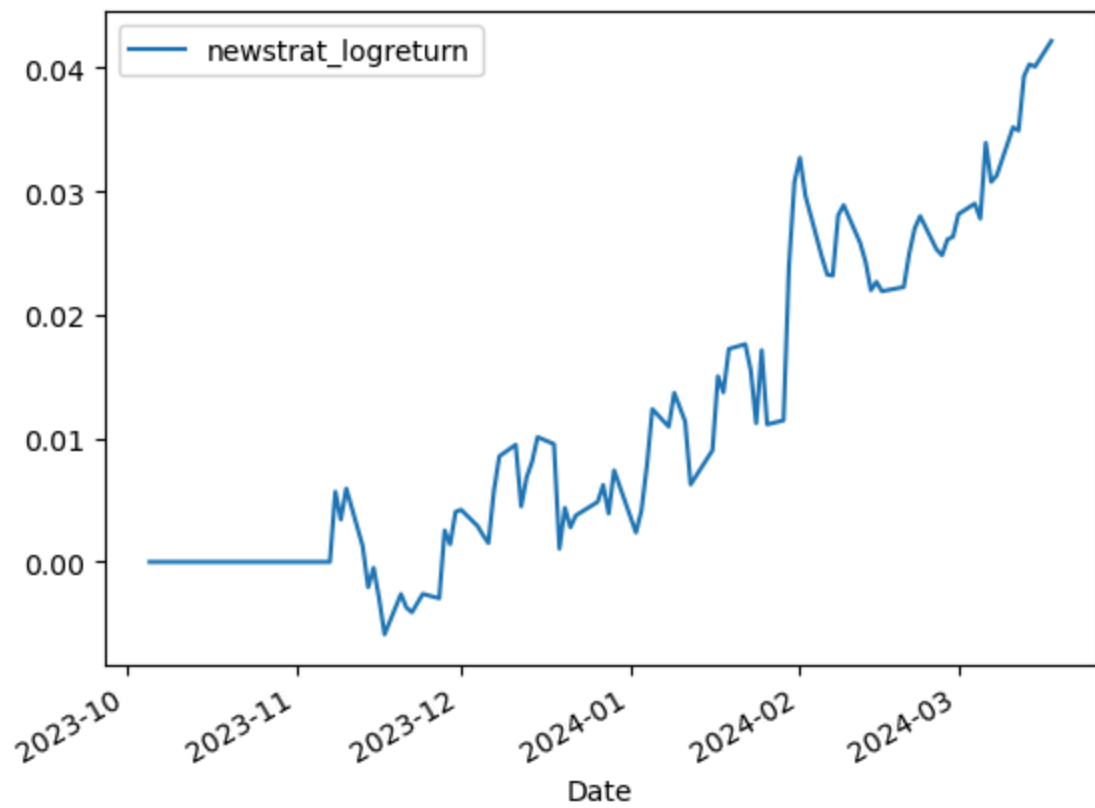
Results

In each of the cells below is an image of the returns of the strategy along with a description of the conditions used.

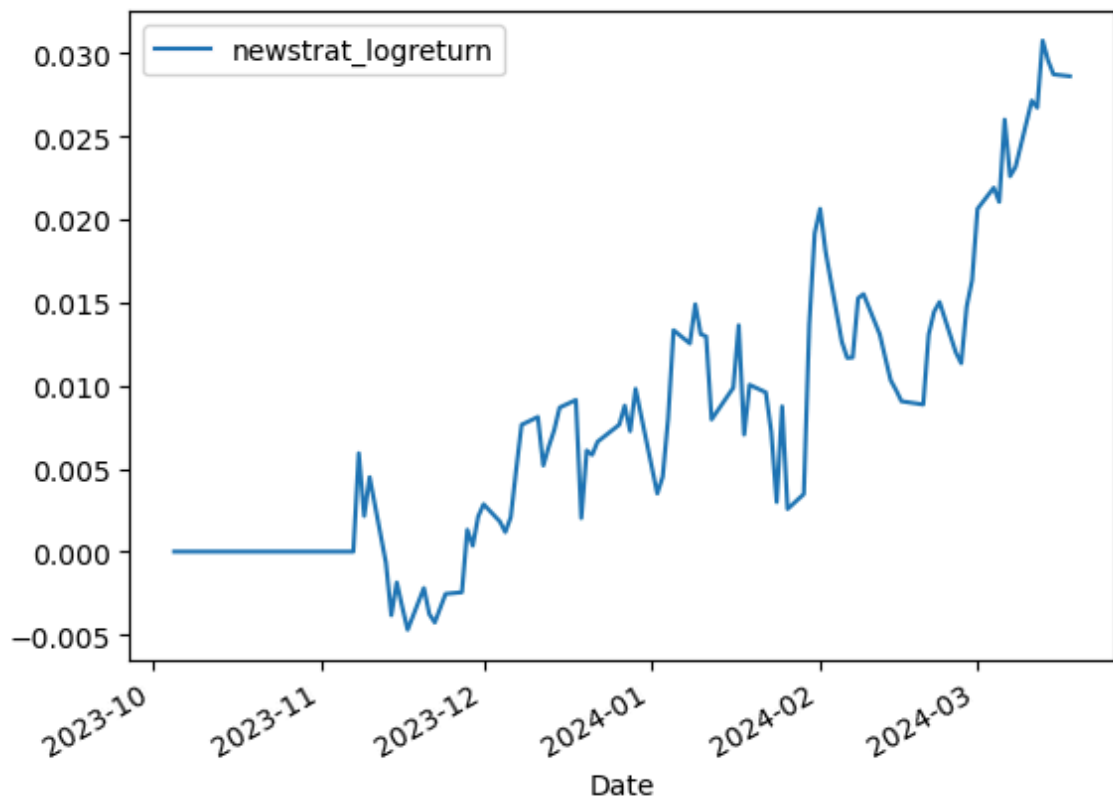
1. In the Fourier analysis in Step 2.1, no frequencies were filtered out. This is virtually the same as using the raw log return data points from before 12 days ago and measuring their ability to predict the data points within the last 12 days.



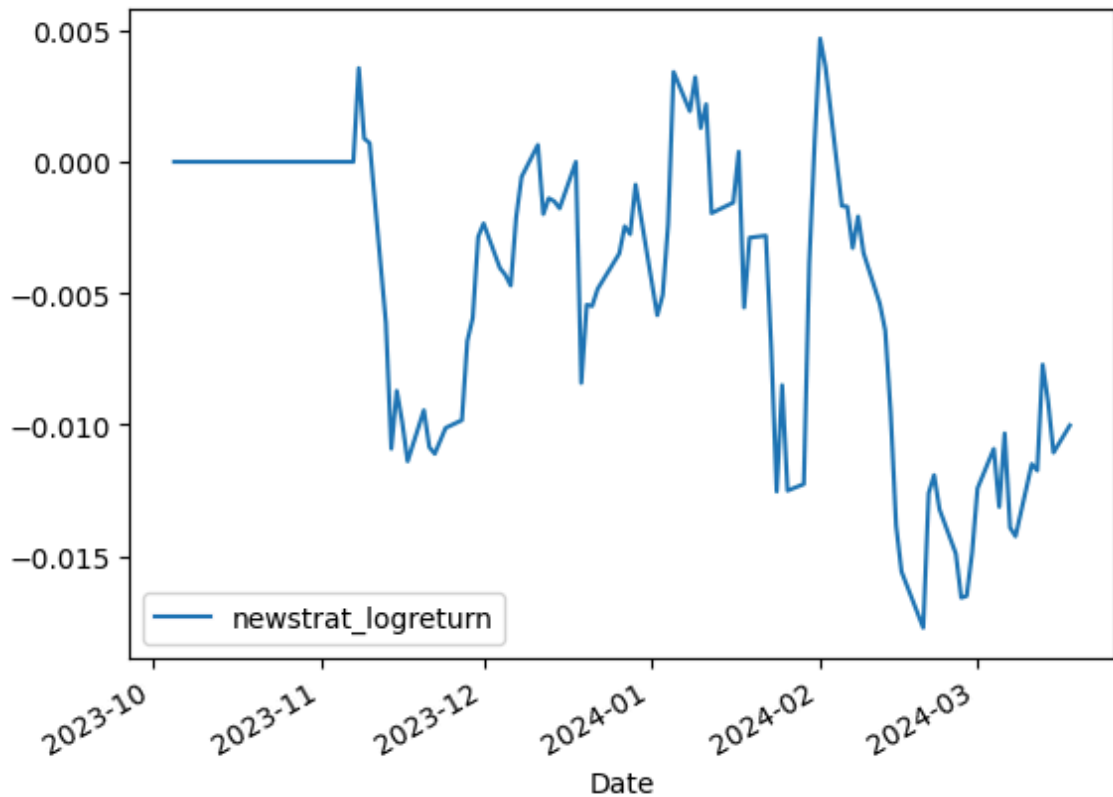
2. In the Fourier analysis in Step 2.1, frequencies that had an amplitude $< 25\%$ than the frequency with the maximum amplitude were filtered out.



3. In the Fourier analysis in Step 2.1, frequencies that had an amplitude $< 50\%$ than the frequency with the maximum amplitude were filtered out.



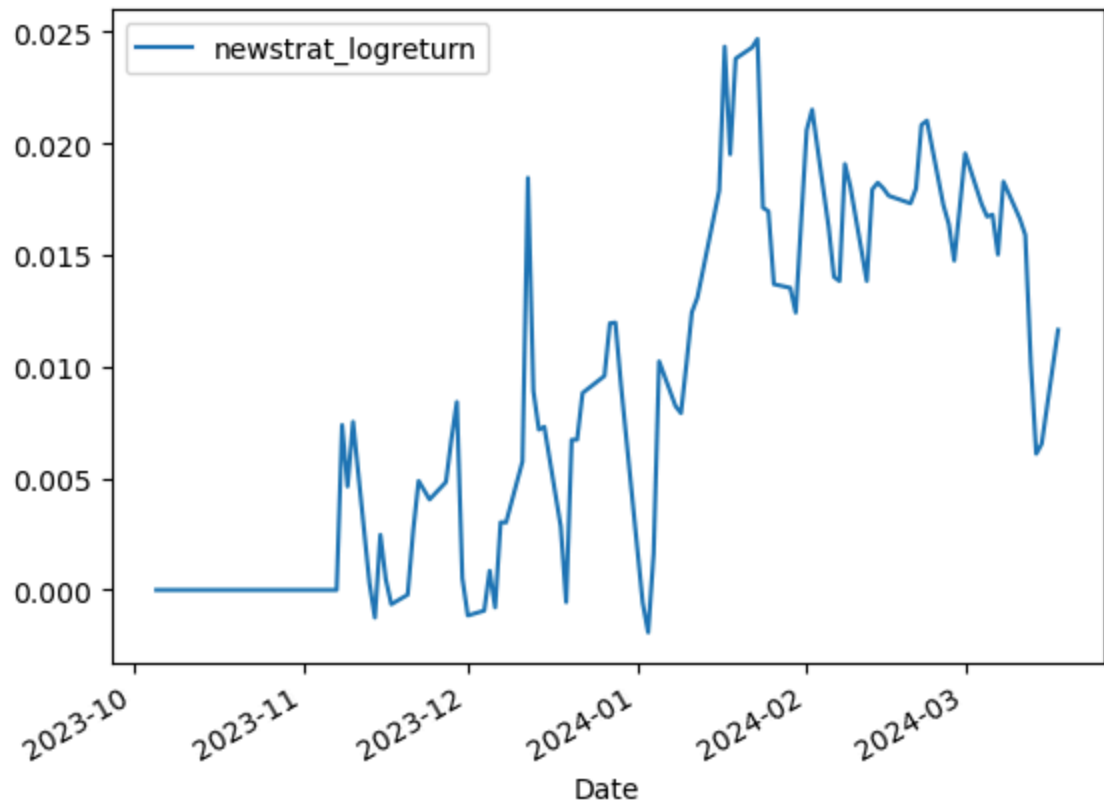
4. In the Fourier analysis in Step 2.1, frequencies that had an amplitude $< 75\%$ than the frequency with the maximum amplitude were filtered out.



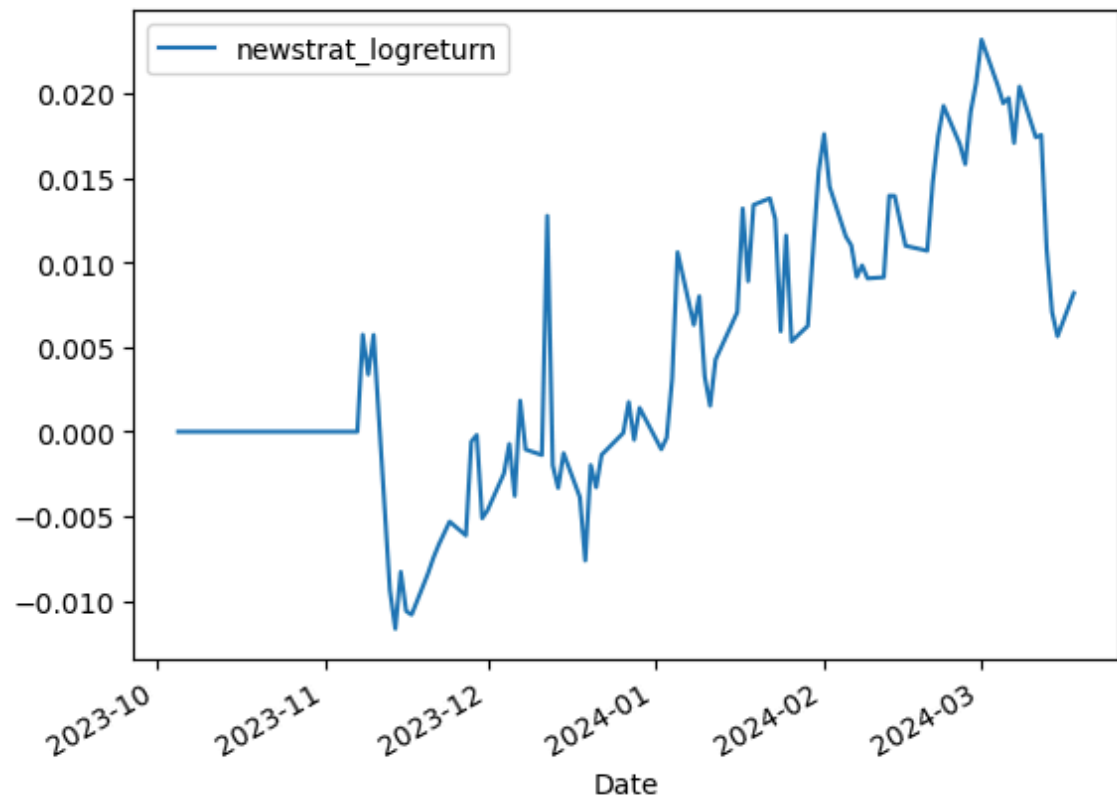
We see that the returns get progressively worse as we filter out more and more frequencies from each selected function in Step 2.1.

I also experimented with Step 1, by trying different ranges of periods for the different periodic functions to test. I initially tried different periodic functions, with periods ranging from 7 days to 12 days. In the following trials, I also tried period ranges of 4 to 7, 5 to 9, 8 to 15, 10 to 18, 11 to 21, 13 to 24, 14 to 27, and 16 to 30. For each of the following trials, no filtering was performed in Step 2.1, as that yielded the best results for the original experiment. Note: Each period range is of the form n to $2n-1$ or n to $2n-2$ because there is no use in testing periods below n . For example, a function that has period $n-1$ also has a period of $2n-2$, which is already included in the period range. Similarly, all functions with smaller periods would have been taken care of by fitting functions with periods in the interval n to $2n-1$ or $2n-2$.

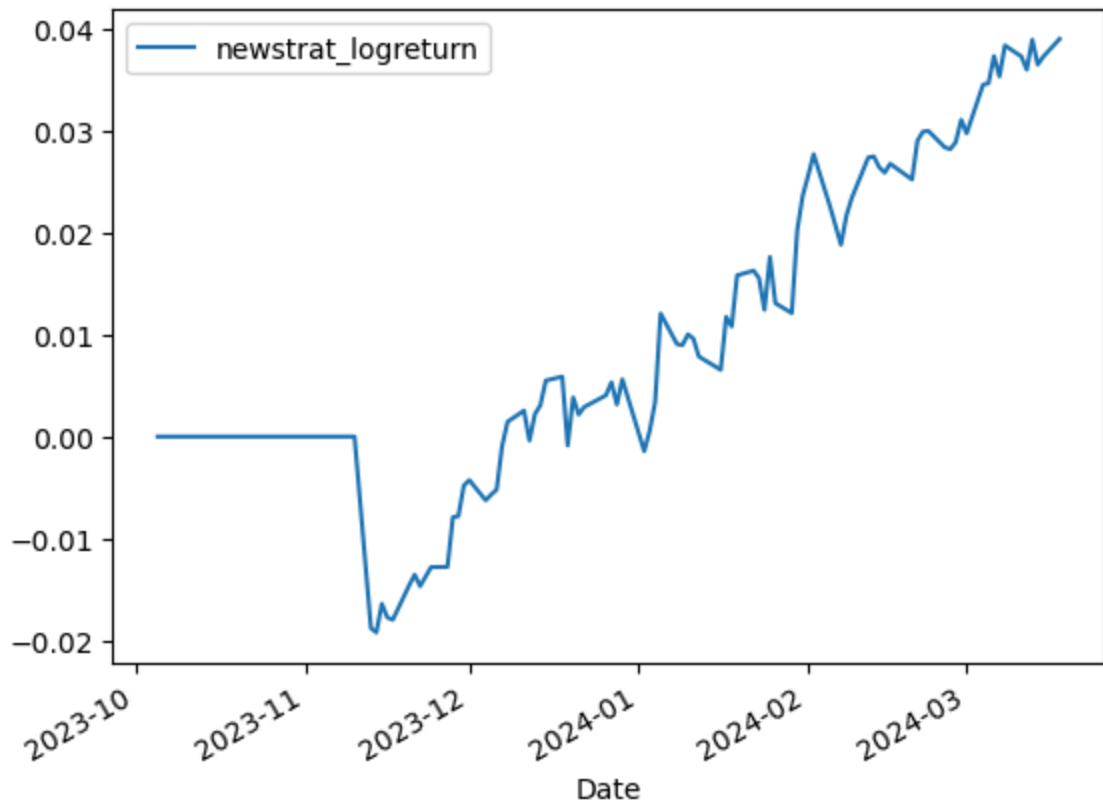
Fitting functions with period of 4 days to 7 days:



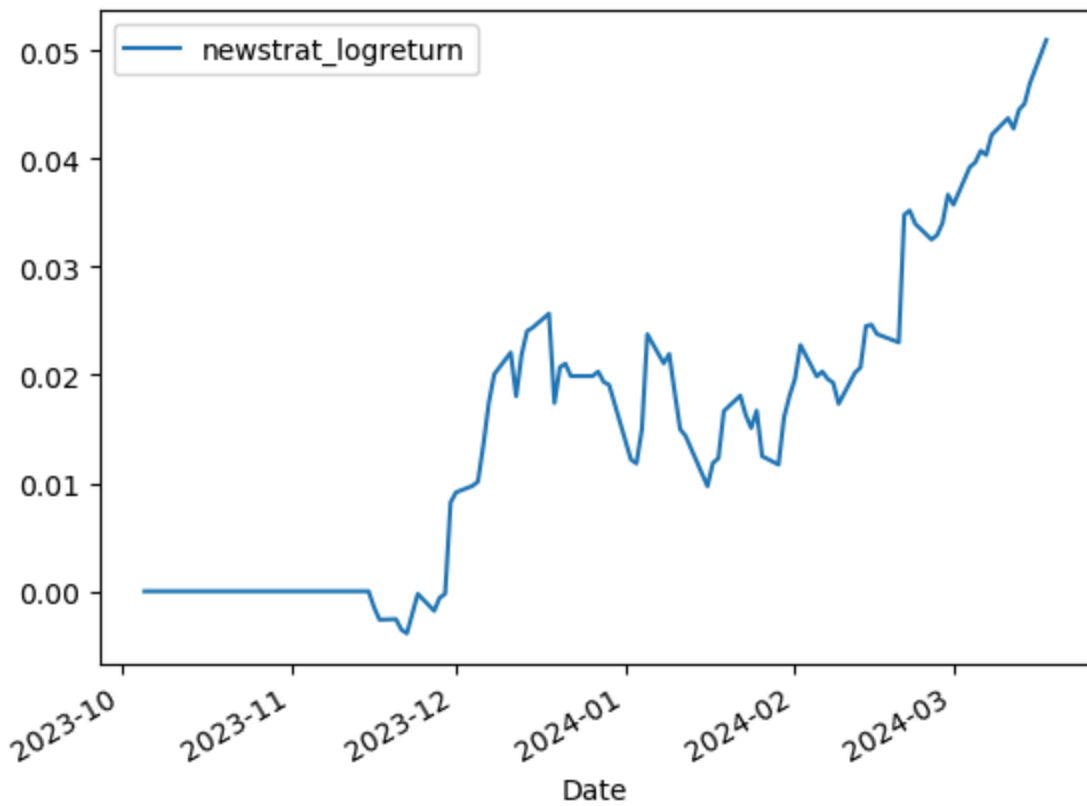
Fitting functions with period of 5 days to 9 days:



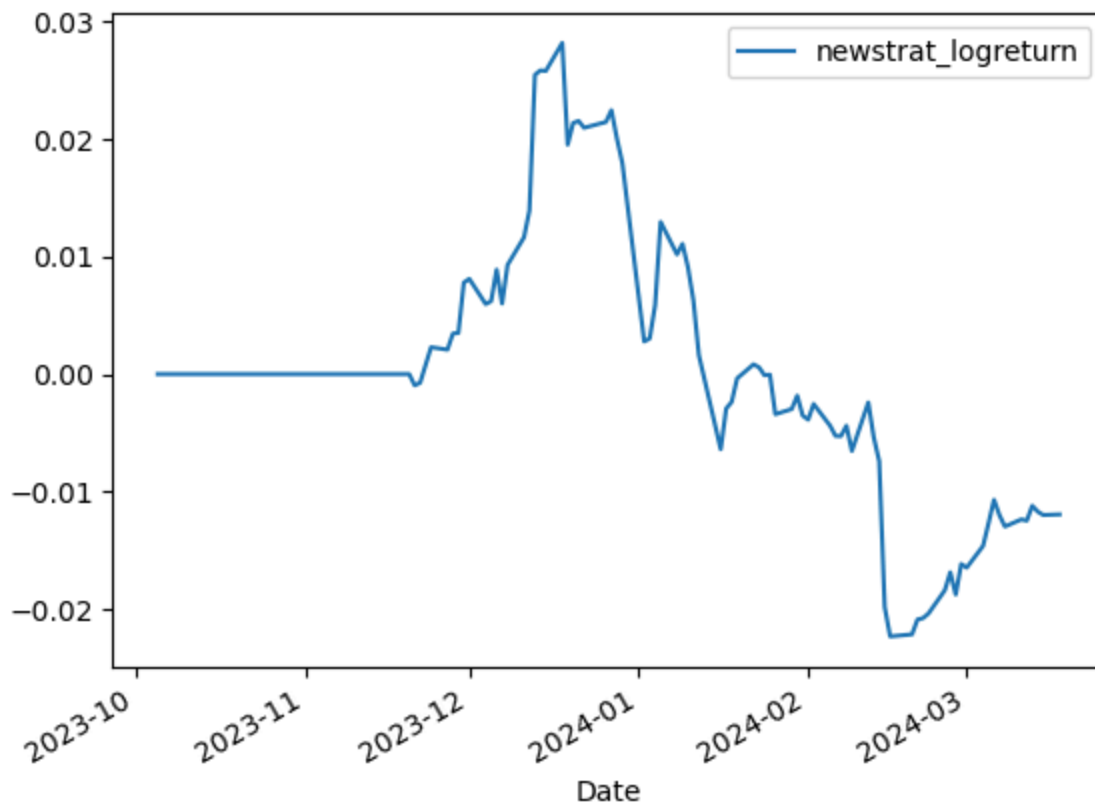
Fitting functions with period of 8 days to 15 days:



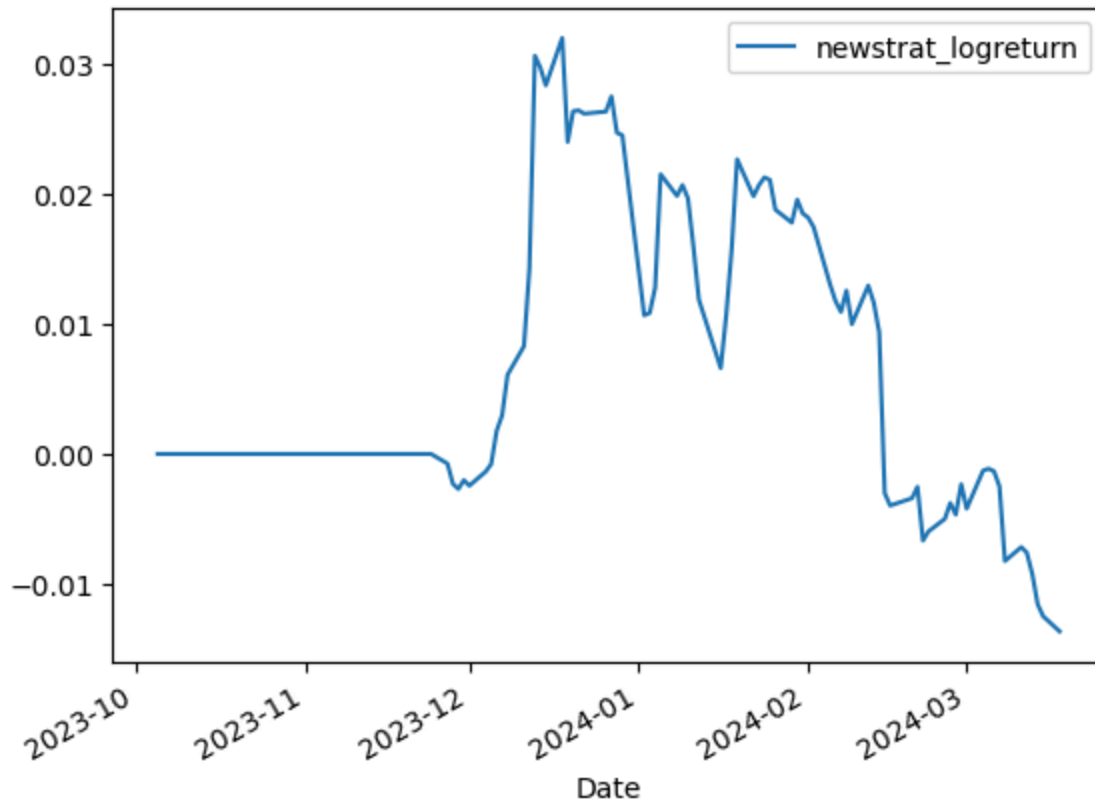
Fitting functions with period of 10 days to 18 days:



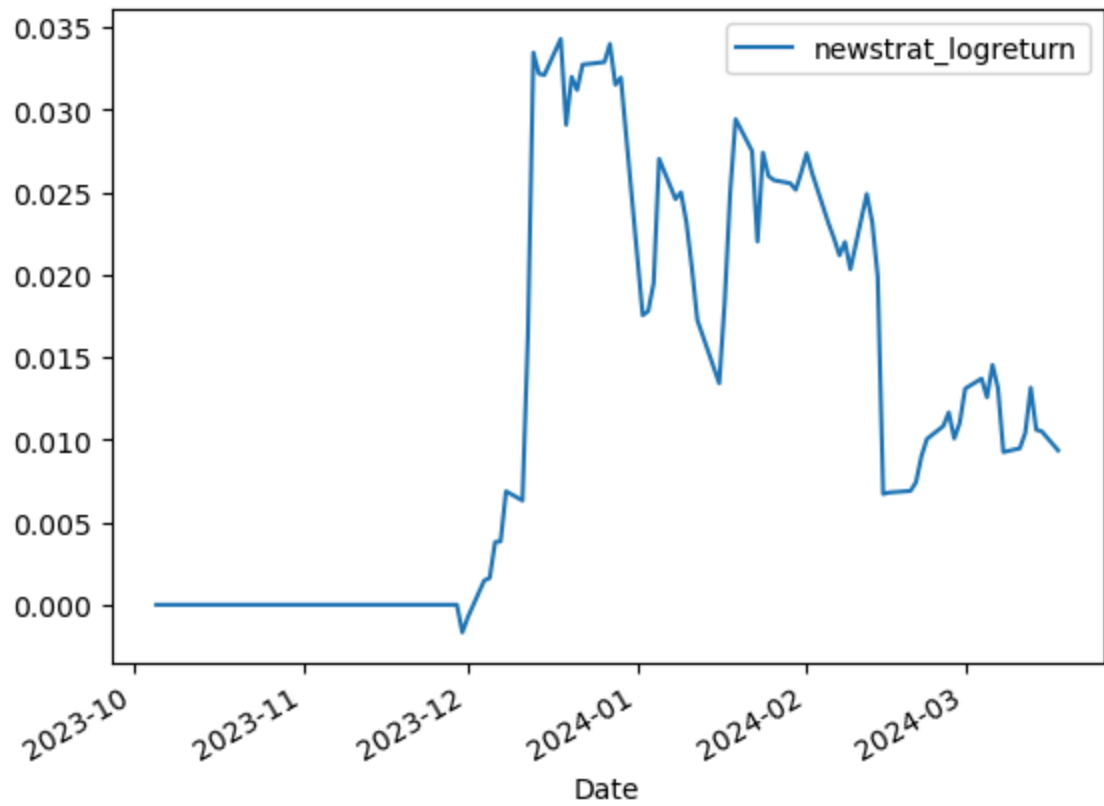
Fitting functions with period of 11 days to 21 days:



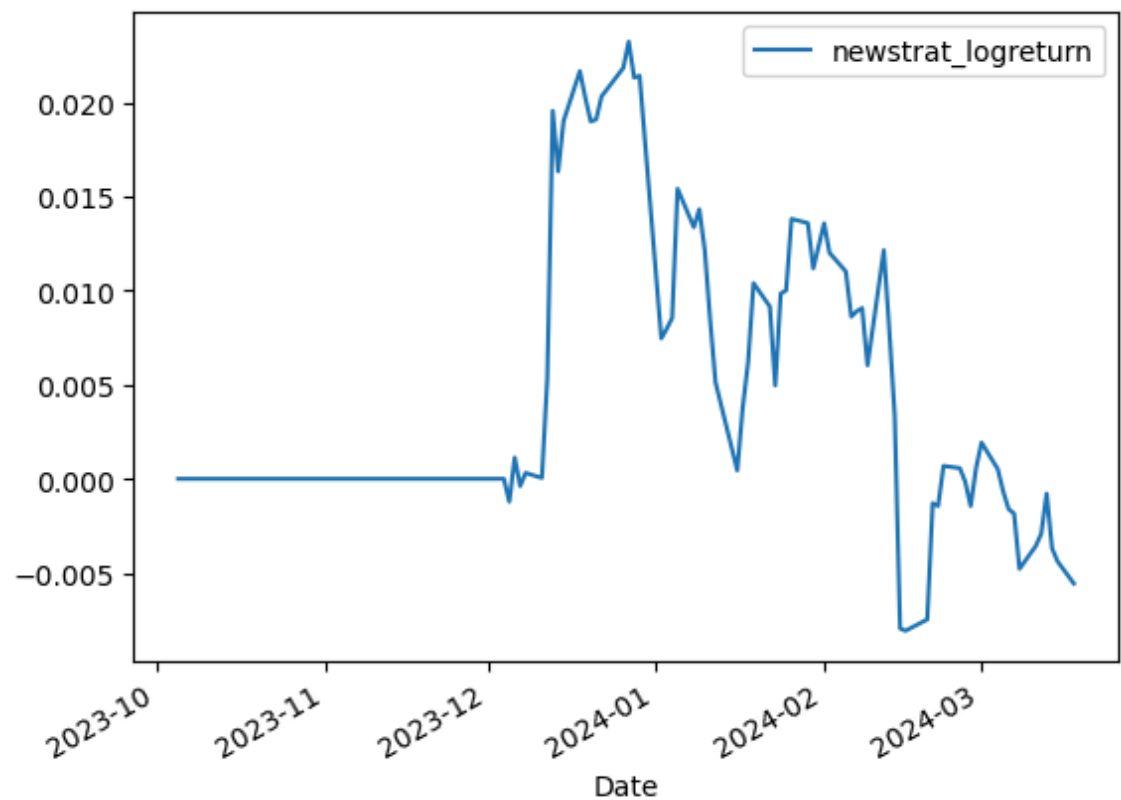
Fitting functions with period of 13 days to 24 days:



Fitting functions with period of 14 days to 27 days:



Fitting functions with period of 16 days to 30 days:



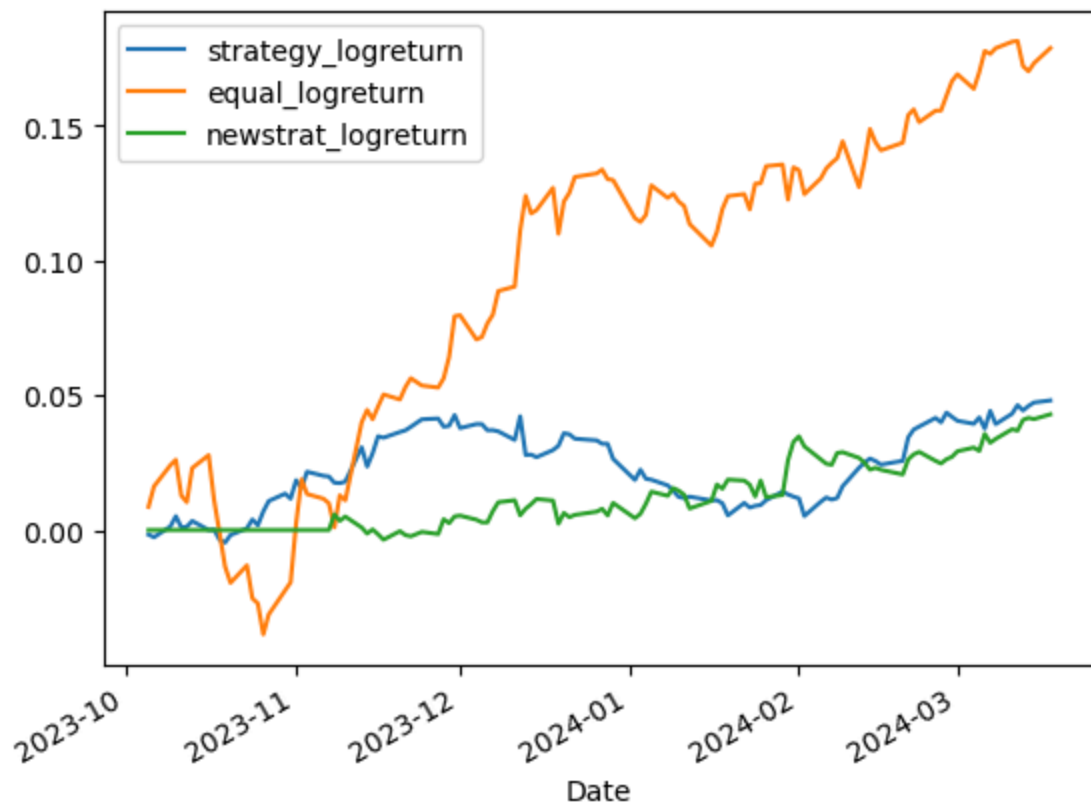
When compared with the original algorithm, log returns for these approaches seem to mostly perform worse and fluctuate wildly. However, using a period range of 8 to 15 gets

us an ending log return close to 0.04, and using a period range of 10 to 18 gets us an ending log return of around 0.05, which is better than for our original period range of 6 to 12. This tells us that taking into account much longer periods, like 18 days or larger, and excluding periods in the range from 6 to 18 causes the algorithm to perform worse and have less predictive power.

Conclusion

In comparison to the momentum based trading strategy shown in Lab 02, this algorithm has a similar ending log return of around 0.04, and the returns seem to fluctuate less for this algorithm.

However, this algorithm doesn't perform nearly as well as the equal position strategy. The image below compares all three strategies, where my strategy is drawn in green.



Please see my code in the section **My Strategy**

```
In [338... # see if you can derive the transformations from log returns to simple retur
hdf = pd.read_parquet('/Users/ishwar/Desktop/IE0R198 Final/stock_data.parque
hdf.head()
```

Out [338...

Price							
Ticker	A	AAL	AAPL	ABBV	ABNB	ABT	ACGL
Date							
2023-10-04	110.917877	12.73	172.585999	141.069550	127.410004	93.262283	76.509293
2023-10-05	109.606766	12.85	173.828293	140.840302	124.989998	93.798546	77.631355
2023-10-06	109.894814	12.76	176.392334	141.594879	126.360001	94.461571	78.144844
2023-10-09	110.530510	12.24	177.883041	142.425888	127.769997	94.344574	78.040245
2023-10-10	112.526970	12.26	177.286758	142.215759	131.589996	95.124588	77.840553

5 rows × 994 columns

In [339...

```
# use np.exp
def logtosimple(logreturn):
    return np.exp(logreturn) - 1
```

In [340...

```
# use np.log
def simpletolog(simplereturn):
    return np.log(simplereturn + 1)
```

In [341...

```
# let's look at an example asset
# create a dataframe that is just the adjusted close and volume for AAL
aal = hdf.loc[:,hdf.columns.get_level_values(1) == "AAL"]
aal.head()
```

Out [341...

Price			Volume
Ticker	AAL	AAL	
Date			
2023-10-04	12.73	49738100	
2023-10-05	12.85	36272900	
2023-10-06	12.76	31822200	
2023-10-09	12.24	51462700	
2023-10-10	12.26	37144200	

Let's remove the top level of the columns in order to make things easier - we already know the symbol is AAL

```
In [342... aal.columns = aal.columns.get_level_values(0)
print(aal.columns)
aal.head()
```

```
Index(['Adj Close', 'Volume'], dtype='object', name='Price')
```

```
Out[342...      Price  Adj Close  Volume
Date
2023-10-04    12.73  49738100
2023-10-05    12.85  36272900
2023-10-06    12.76  31822200
2023-10-09    12.24  51462700
2023-10-10    12.26  37144200
```

To calculate returns for **AAL**, create a new column by applying `.pct_change()` to the **adjusted close** column.

```
In [343... aal['returns'] = aal["Adj Close"].pct_change()
aal.head()
```

```
Out[343...      Price  Adj Close  Volume  returns
Date
2023-10-04    12.73  49738100      NaN
2023-10-05    12.85  36272900  0.009427
2023-10-06    12.76  31822200 -0.007004
2023-10-09    12.24  51462700 -0.040752
2023-10-10    12.26  37144200  0.001634
```

```
In [344... # we can get the log returns by adjusting the percent change function

aal['logreturns'] = aal['returns'].apply(simpletolog)
aal.head()
```

Out [344...

	Price	Adj Close	Volume	returns	logreturns
Date					
2023-10-04	12.73	49738100		NaN	NaN
2023-10-05	12.85	36272900		0.009427	0.009382
2023-10-06	12.76	31822200		-0.007004	-0.007029
2023-10-09	12.24	51462700		-0.040752	-0.041606
2023-10-10	12.26	37144200		0.001634	0.001633

In [345...

```
# .cumsum() on a series will add everything in the column up until that point
# this will give us the log return from the start, to the current index
aal['cum_logreturns'] = aal["logreturns"].cumsum()
aal.head()
```

Out [345...

	Price	Adj Close	Volume	returns	logreturns	cum_logreturns
Date						
2023-10-04	12.73	49738100		NaN	NaN	NaN
2023-10-05	12.85	36272900		0.009427	0.009382	0.009382
2023-10-06	12.76	31822200		-0.007004	-0.007029	0.002354
2023-10-09	12.24	51462700		-0.040752	-0.041606	-0.039252
2023-10-10	12.26	37144200		0.001634	0.001633	-0.037619

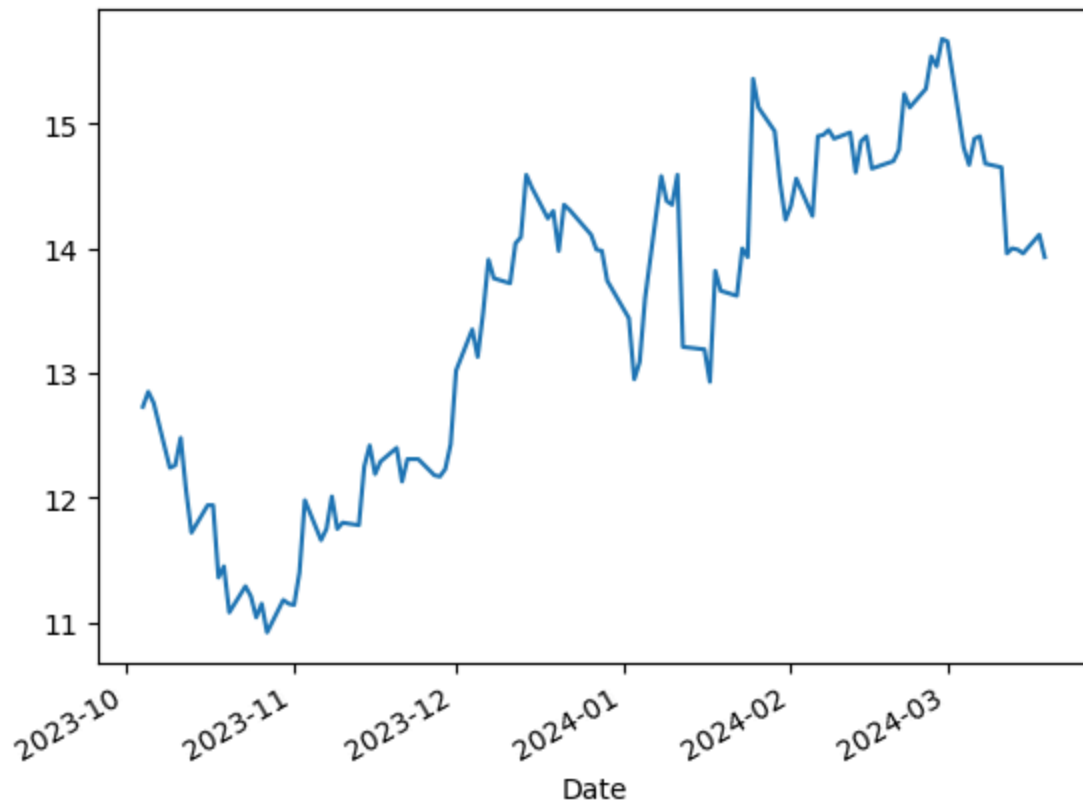
In [346...

```
# pandas also comes equipped with some built in plotting
# we can plot the adjusted close, as well as the logreturns cumsum, and see
print(len(aal['Adj Close']))
aal['Adj Close'].plot()
```

115

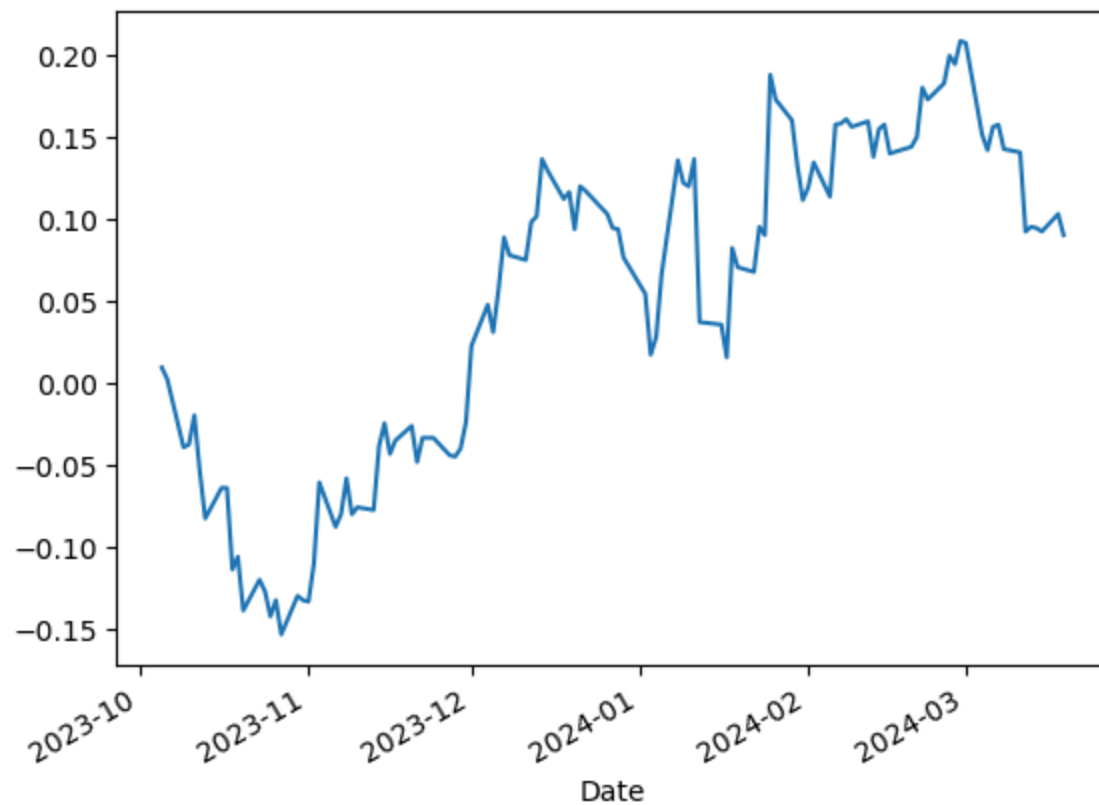
Out [346...

<Axes: xlabel='Date'>



```
In [347... aal['cum_logreturns'].plot()
```

```
Out[347... <Axes: xlabel='Date'>
```



Basic trend strategy: long/short by percentile

Developing a Trading Strategy

Now, let's work on creating a **trading strategy** with the following assumptions:

- **Shorting Allowed:** We assume that we can short stocks.
- **Zero Borrowing Cost:** For simplicity, we assume the cost to borrow is **0** (note: this is a non-trivial assumption).

Strategy Overview

Our goal is to:

- **Buy (go long)** stocks expected to increase in value.
- **Sell (go short)** stocks expected to decrease in value.

Approach

1. Ranking Stocks:

We will rank stocks based on a chosen **metric** (e.g., momentum, valuation ratios, or other factors).

2. Position Allocation:

- Go **long** on the top percentage of stocks ranked by the metric.
- Go **short** on the bottom percentage of stocks ranked by the metric.

This approach allows us to systematically identify opportunities and construct a portfolio based on clear criteria.

Out [348...

Price							
Ticker	A	AAL	AAPL	ABBV	ABNB	ABT	ACI
Date							
2023-10-04	110.917877	12.73	172.585999	141.069550	127.410004	93.262283	76.50929
2023-10-05	109.606766	12.85	173.828293	140.840302	124.989998	93.798546	77.63139
2023-10-06	109.894814	12.76	176.392334	141.594879	126.360001	94.461571	78.14484
2023-10-09	110.530510	12.24	177.883041	142.425888	127.769997	94.344574	78.04024
2023-10-10	112.526970	12.26	177.286758	142.215759	131.589996	95.124588	77.84059
2023-10-11	112.616356	12.48	178.688049	142.645569	130.000000	90.327415	78.94359
2023-10-12	108.206268	12.06	179.592422	143.070053	125.970001	88.425377	78.97219
2023-10-13	109.765694	11.72	177.743927	142.742065	124.080002	89.092072	80.02769
2023-10-16	110.749023	11.94	177.614700	142.037796	125.190002	90.376442	81.49209
2023-10-17	111.642975	11.94	176.054413	143.919037	125.709999	90.337219	82.36689
2023-10-18	107.878494	11.36	174.752518	144.015503	122.169998	93.690300	81.25429
2023-10-19	108.772430	11.45	174.374878	140.388107	119.970001	93.572655	79.66629
2023-10-20	108.494316	11.08	171.810822	141.073044	116.489998	94.886436	78.69639
2023-10-23	108.603577	11.29	171.930084	139.625961	120.389999	93.905998	77.84059
2023-10-24	104.928490	11.21	172.367386	141.150238	122.010002	92.954979	79.32399
2023-10-25	102.703583	11.04	170.041855	140.137253	118.500000	91.739235	79.39059
2023-10-26	103.607452	11.15	165.857880	140.079391	115.470001	92.141220	78.40159
2023-10-27	102.077820	10.92	167.179657	134.030502	114.089996	91.033325	77.36519
2023-10-30	100.488594	11.18	169.236847	136.886108	116.949997	91.180397	78.81049

Price							
Ticker	A	AAL	AAPL	ABBV	ABNB	ABT	ACI
Date							
2023-10-31	102.673782	11.15	169.713882	136.201141	118.290001	92.700066	82.42381
2023-11-01	102.167213	11.14	172.894104	137.445663	119.470001	93.151070	85.58086
2023-11-02	103.766373	11.40	176.471832	138.207794	115.500000	94.082474	81.72021
2023-11-03	108.285721	11.98	175.557526	136.432678	122.639999	93.964828	81.34937
2023-11-06	106.805763	11.66	178.121552	136.220428	118.169998	93.141258	80.97851
2023-11-07	109.795486	11.75	180.695557	137.146606	121.080002	92.945183	81.21624
2023-11-08	108.653236	12.01	181.758926	137.021164	117.660004	92.886345	79.98951
2023-11-09	107.014343	11.75	181.281906	133.171875	115.519997	91.719635	80.51257
2023-11-10	107.739441	11.80	185.491272	133.702469	118.150002	92.131424	81.72971
2023-11-13	106.736237	11.78	183.899063	133.750732	119.150002	93.915810	82.31921
2023-11-14	110.858292	12.25	186.526199	133.191193	126.680000	95.180557	81.30181
2023-11-15	112.834877	12.42	187.093430	132.747406	128.350006	96.082565	78.60127
2023-11-16	113.420906	12.19	188.785141	133.403427	126.279999	98.298355	79.59977
2023-11-17	112.387917	12.29	188.765244	133.422729	127.150002	97.602242	79.49511
2023-11-20	113.212334	12.40	190.516647	133.432358	129.699997	99.219963	80.31281
2023-11-21	123.085373	12.13	189.710602	133.837555	126.820000	99.916069	81.90081
2023-11-22	123.154900	12.31	190.377335	133.586716	129.039993	100.690605	81.72971
2023-11-24	125.767189	12.31	189.043869	133.779663	128.369995	100.857277	82.34781
2023-11-27	124.168037	12.18	188.864746	134.184860	129.000000	100.700401	82.72811

Price							
Ticker	A	AAL	AAPL	ABBV	ABNB	ABT	ACI
Date							
2023-11-28	123.870049	12.17	189.471771	133.210480	127.559998	100.063133	80.3033
2023-11-29	126.730652	12.23	188.446793	133.615677	126.480003	101.602417	78.3350
2023-11-30	126.939255	12.43	189.023972	137.368484	126.339996	102.249496	79.58070
2023-12-01	127.922569	13.02	190.307678	138.352493	135.020004	102.827950	78.57274
2023-12-04	128.011978	13.35	188.506500	139.066406	133.699997	103.131897	77.96410
2023-12-05	127.018700	13.13	192.477051	139.471619	133.710007	102.308319	77.83105
2023-12-06	128.041779	13.48	191.382431	141.217773	135.309998	102.886780	76.04339
2023-12-07	127.813316	13.91	193.322906	142.751694	139.839996	102.014191	74.49339
2023-12-08	126.343277	13.76	194.755890	144.015503	140.679993	102.465195	75.40629
2023-12-11	128.101364	13.72	192.238205	145.906387	142.910004	104.141731	75.70105
2023-12-12	127.922569	14.04	193.760773	147.835846	140.550003	104.592735	76.33815
2023-12-13	132.839233	14.09	196.994919	148.858459	144.509995	105.151581	74.53145
2023-12-14	137.030807	14.59	197.144165	149.418015	147.259995	106.475166	71.00359
2023-12-15	135.858765	14.49	196.606812	148.607620	146.600006	105.190796	69.81499
2023-12-18	136.603714	14.24	194.935013	148.009491	147.500000	105.945740	70.98459
2023-12-19	138.848465	14.30	195.979889	148.183151	146.910004	106.298691	71.15579
2023-12-20	137.249344	13.98	193.880188	146.340515	141.070007	105.298645	69.99569
2023-12-21	138.004211	14.35	193.730881	147.208771	142.089996	106.357521	70.51869
2023-12-22	138.629974	14.31	192.656174	149.475891	140.800003	106.896759	70.21439

Price							
Ticker	A	AAL	AAPL	ABBV	ABNB	ABT	ACI
Date							
2023-12-26	138.868347	14.11	192.108856	149.167175	138.720001	107.092842	69.9005
2023-12-27	138.878296	13.99	192.208359	149.418015	136.550003	107.641884	69.9290
2023-12-28	138.828629	13.98	192.636276	149.292587	137.000000	108.239952	70.42354
2023-12-29	138.327164	13.74	191.591385	149.504822	136.139999	107.916412	70.62320
2024-01-02	138.048584	13.44	184.734970	154.183807	134.479996	107.700714	71.84038
2024-01-03	130.496964	12.95	183.351746	154.801224	133.419998	107.377167	73.16210
2024-01-04	130.337769	13.09	181.023178	155.765961	133.720001	108.808609	73.66610
2024-01-05	129.899994	13.60	180.296707	156.421967	135.979996	108.632118	73.0385
2024-01-08	132.705719	14.58	184.655365	155.737000	140.080002	110.200829	73.33329
2024-01-09	130.019363	14.38	184.237411	156.585983	139.529999	110.524361	72.4109
2024-01-10	130.427292	14.35	185.282303	159.074997	139.759995	112.161690	73.18110
2024-01-11	129.024429	14.59	184.685226	158.004150	139.449997	111.816879	73.80874
2024-01-12	129.880081	13.21	185.013611	158.169708	137.139999	112.230652	73.87530
2024-01-16	129.870132	13.19	182.734772	157.273682	135.020004	112.073029	74.33170
2024-01-17	127.472313	12.93	181.789383	157.819077	134.979996	112.240501	74.80710
2024-01-18	129.800507	13.82	187.710419	159.046265	137.039993	112.329163	75.08290
2024-01-19	130.556656	13.66	190.626114	160.477982	139.929993	112.989243	76.90860
2024-01-22	131.879929	13.62	192.944748	161.081818	142.009995	112.496643	76.51880
2024-01-23	130.974533	14.00	194.228470	163.136856	142.300003	112.309471	76.39510

Price							
Ticker	A	AAL	AAPL	ABBV	ABNB	ABT	ACI
Date							
2024-01-24	129.123932	13.93	193.551773	159.942307	141.190002	109.127357	76.19549
2024-01-25	129.223419	15.36	193.223404	160.828613	142.110001	110.782455	75.6915
2024-01-26	130.327835	15.13	191.481918	160.117599	149.619995	110.348976	75.9767
2024-01-29	132.158524	14.94	190.795288	159.640366	152.699997	111.767616	75.81514
2024-01-30	133.601181	14.53	187.123260	160.624069	149.440002	111.797188	76.62340
2024-01-31	129.442322	14.23	183.501022	160.117599	144.139999	111.472069	78.38250
2024-02-01	132.377396	14.34	185.949051	163.224518	146.490005	112.792198	78.05929
2024-02-02	132.297806	14.56	184.943954	164.276382	146.550003	110.447495	79.20984
2024-02-05	132.148575	14.26	186.765030	166.798920	144.779999	109.994316	79.5521
2024-02-06	135.173187	14.90	188.377136	168.776047	144.529999	112.309471	79.20984
2024-02-07	132.118713	14.91	188.486603	170.451248	147.550003	111.629700	80.27484
2024-02-08	133.392258	14.95	187.401917	170.236969	150.539993	110.762749	78.69630
2024-02-09	132.665924	14.88	188.169144	169.545456	147.595001	110.151939	79.36199
2024-02-12	131.641144	14.93	186.475266	168.561783	153.800003	110.861267	79.01960
2024-02-13	129.104034	14.61	184.372864	168.776047	150.820007	109.688904	79.93250
2024-02-14	131.571503	14.86	183.486069	169.876602	148.199997	110.398232	80.70279
2024-02-15	134.068802	14.90	183.197128	171.990082	157.690002	111.954803	83.10850

Price							
Ticker	A	AAL	AAPL	ABBV	ABNB	ABT	ACI
Date							
2024-02-16	134.158356	14.64	181.652695	172.866638	152.509995	112.319321	81.8818
2024-02-20	133.461884	14.70	180.905411	171.171967	148.149994	114.910316	81.1782
2024-02-21	134.645889	14.79	181.662689	170.577850	149.240005	116.122086	80.5506
2024-02-22	131.352600	15.24	183.705261	172.145920	155.259995	117.255013	81.6061
2024-02-23	131.362549	15.13	181.861969	173.451004	152.660004	117.688492	83.0039
2024-02-26	130.825302	15.28	180.506866	174.152267	149.270004	116.929916	82.9944
2024-02-27	131.879929	15.54	181.971558	174.327560	152.059998	117.629395	82.9373
2024-02-28	136.317383	15.46	180.765915	173.363342	153.429993	118.269745	83.9453
2024-02-29	136.665619	15.68	180.098328	171.464157	157.470001	116.880653	83.2892
2024-03-01	138.357025	15.66	179.012268	174.249649	159.720001	116.860954	82.8898
2024-03-04	142.137802	14.81	174.468704	172.438110	158.089996	118.259903	82.8612
2024-03-05	143.620285	14.67	169.506653	174.493149	159.330002	116.723038	83.0990
2024-03-06	145.480820	14.88	168.510269	176.343643	163.869995	117.570274	84.4588
2024-03-07	148.555191	14.90	168.390701	175.866409	163.539993	119.126839	83.3842
2024-03-08	147.122482	14.68	170.114456	174.191223	164.910004	119.166260	83.3082
2024-03-11	146.545410	14.65	172.127182	174.950912	162.990005	118.407677	85.8756

Price								
Ticker	A	AAL	AAPL	ABBV	ABNB	ABT	ACI	
Date								
2024-03-12	147.092636	13.96	172.605438	176.207291	166.669998	118.969223	86.45569	
2024-03-13	147.072739	14.00	170.513031	175.174911	164.759995	118.378120	86.53176	
2024-03-14	144.475922	13.99	172.376282	176.479996	166.440002	117.087540	86.56029	
2024-03-15	146.734451	13.96	171.997650	173.246490	160.639999	113.777367	87.36851	
2024-03-18	145.351471	14.11	173.093674	173.840591	161.860001	110.772606	86.91211	
2024-03-19	146.644913	13.93	175.445190	174.980118	161.789993	111.481926	87.16886	

115 rows x 994 columns

```
In [349... # to more easily group by asset, we'll make assets into its own column using
hdf_old = hdf.copy()
hdf = (
    hdf.stack()
    .reset_index()
    .rename(columns={'level_1': 'Symbol'})
)
hdf
```

Out [349...

	Price	Date	Ticker	Adj Close	Volume
0	2023-10-04	A	110.917877	1541700.0	
1	2023-10-04	AAL	12.730000	49738100.0	
2	2023-10-04	AAPL	172.585999	53020300.0	
3	2023-10-04	ABBV	141.069550	3159600.0	
4	2023-10-04	ABNB	127.410004	5822000.0	
...	...	...	...	...	
57150	2024-03-19	XYL	127.775208	806900.0	
57151	2024-03-19	YUM	135.940338	2266500.0	
57152	2024-03-19	ZBH	125.862167	949300.0	
57153	2024-03-19	ZBRA	285.190002	306000.0	
57154	2024-03-19	ZTS	171.652222	3624000.0	

57155 rows × 4 columns

Calculating Returns and Log Returns

To compute **returns** and **log returns** for the entire dataset, we can **group by the ticker**, select the adjusted close column, and then use `.pct_change()` to get the percentage change from one row to the next

These steps will provide the necessary data to derive both simple and log returns for analysis.

```
In [350... hdf.groupby('Ticker')['Adj Close'].pct_change()
```

```
Out [350... 0      NaN
1      NaN
2      NaN
3      NaN
4      NaN
...
57150    0.007530
57151    0.007485
57152    0.005557
57153   -0.002728
57154    0.013299
Name: Adj Close, Length: 57155, dtype: float64
```

```
In [351... hdf['returns'] = hdf.groupby('Ticker')['Adj Close'].pct_change()
hdf.head()
```

Out[351...	Price	Date	Ticker	Adj Close	Volume	returns
	0	2023-10-04	A	110.917877	1541700.0	NaN
	1	2023-10-04	AAL	12.730000	49738100.0	NaN
	2	2023-10-04	AAPL	172.585999	53020300.0	NaN
	3	2023-10-04	ABBV	141.069550	3159600.0	NaN
	4	2023-10-04	ABNB	127.410004	5822000.0	NaN

Creating a Log Returns Column

To calculate log returns, we'll create a new column in the dataset. This involves applying our **simple-to-log** function to the data and assigning the results to a column named `logreturns`.

This new column will represent the log-transformed returns for each data point.

```
In [352... hdf['logreturns'] = hdf['returns'].apply(simpletolog)
hdf.head()
```

Out [352...	Price	Date	Ticker	Adj Close	Volume	returns	logreturns
	0	2023-10-04	A	110.917877	1541700.0	NaN	NaN
	1	2023-10-04	AAL	12.730000	49738100.0	NaN	NaN
	2	2023-10-04	AAPL	172.585999	53020300.0	NaN	NaN
	3	2023-10-04	ABBV	141.069550	3159600.0	NaN	NaN
	4	2023-10-04	ABNB	127.410004	5822000.0	NaN	NaN

Creating a Forward Log Return Column

To predict the next step of returns, we need to create a column for the **forward log return**. Here's how to do it:

1. **Group by Ticker:** Group the data by the stock ticker to ensure calculations are performed within each stock's data.
2. **Select Log Returns:** Focus on the column containing the log returns.
3. **Shift the Series:** Use `.shift(-1)` to move the series **up one row**, aligning the future return with the current row.
Refer to the [pandas.DataFrame.shift documentation](#) for more details.

This process creates a new column that represents the **forward log return**, which can be used as the target variable for prediction models.


```
In [353... hdf['fwd_logreturn'] = hdf.groupby('Ticker')['logreturns'].shift(-1)
hdf.head()
```

```
Out[353... Price    Date    Ticker    Adj Close    Volume    returns    logreturns    fwd_logreturn
0    2023-10-04    A    110.917877    1541700.0    NaN    NaN    -0.011891
1    2023-10-04    AAL    12.730000    49738100.0    NaN    NaN    0.009382
2    2023-10-04    AAPL    172.585999    53020300.0    NaN    NaN    0.007172
3    2023-10-04    ABBV    141.069550    3159600.0    NaN    NaN    -0.001626
4    2023-10-04    ABNB    127.410004    5822000.0    NaN    NaN    -0.019177
```

Dropping Rows with NaN Values

To avoid issues in future operations, we need to drop rows where **log returns** or **forward log returns** contain `NaN` values. This can be done using the `subset` parameter in `df.dropna(subset=[])`

```
In [354... hdf = hdf.dropna(subset=['logreturns', 'fwd_logreturn'])
hdf.head()
```

```
Out[354... Price    Date    Ticker    Adj Close    Volume    returns    logreturns    fwd_logreturn
497    2023-10-05    A    109.606766    1295700.0    -0.011821    -0.011891    0.002625
498    2023-10-05    AAL    12.850000    36272900.0    0.009427    0.009382    -0.007029
499    2023-10-05    AAPL    173.828293    48527900.0    0.007198    0.007172    0.014643
500    2023-10-05    ABBV    140.840302    3599500.0    -0.001625    -0.001626    0.005343
501    2023-10-05    ABNB    124.989998    7630200.0    -0.018994    -0.019177    0.010901
```

Momentum

Ranking Returns by Asset

To develop a strategy, we can **rank the returns** for each asset. But first, we need to decide:

What metric should we use for ranking?

Observations:

- **Momentum Tendency:**

We might guess that stocks that have **gone up** tend to continue rising, while stocks that have **gone down** tend to keep falling.

Why Might This Happen?

This behavior makes economic sense:

- A stock that has risen significantly might attract buyers who want to **join the trend**, driving the price higher.
- Conversely, a stock that has fallen may face additional selling pressure from investors exiting their positions.

Strategy:

- **Buy (Go Long):** Identify and buy the stocks that have risen the most.
 - **Sell (Go Short):** Identify and short the stocks that have fallen the most.
-

Calculating Ranks Using `.rank()`

To calculate ranks for log returns, we can use the `.rank()` method. This method assigns a rank value as if the DataFrame were sorted. Key steps:

1. **Descending Order:** Use `ascending=False` to rank in descending order.
2. **Dense Ranking:** Specify `method='dense'` to ensure that ranks are consecutive even when there are ties.
3. **Group by Date:** Group the data by date to calculate ranks within each group.
4. **Assign to a New Column:** Create a new column, `logreturn_rank`, to store the ranks of the `logreturns` column.

Refer to the [pandas.DataFrame.rank documentation](#) for more details.

```
In [355... hdf['logreturn_rank'] = hdf.groupby('Date')['logreturns'].rank(ascending=False)
hdf.head()
```

Out [355...	Price	Date	Ticker	Adj Close	Volume	returns	logreturns	fwd_logreturn	I
	497	2023-10-05	A	109.606766	1295700.0	-0.011821	-0.011891	0.002625	
	498	2023-10-05	AAL	12.850000	36272900.0	0.009427	0.009382	-0.007029	
	499	2023-10-05	AAPL	173.828293	48527900.0	0.007198	0.007172	0.014643	
	500	2023-10-05	ABBV	140.840302	3599500.0	-0.001625	-0.001626	0.005343	
	501	2023-10-05	ABNB	124.989998	7630200.0	-0.018994	-0.019177	0.010901	

In [356...

```
pd.set_option('display.max_rows', 200)
#hdf.groupby('Ticker').get_group("AAL")
hdf.groupby('Date').get_group("2023-10-05").sort_values(by=['logreturn_rank'
```

Out [356...	Price	Date	Ticker	Adj Close	Volume	returns	logreturns	fwd_logreturn	I
	786	2023-10-05	LW	95.649353	7554600.0	0.080230	0.077174	-0.044773	
	804	2023-10-05	MKTX	222.038803	787600.0	0.057811	0.056202	0.056525	
	613	2023-10-05	CSGP	79.489998	2578700.0	0.040037	0.039256	0.014364	
	663	2023-10-05	EQT	40.214714	6080000.0	0.036318	0.035674	0.040540	
	835	2023-10-05	NRG	37.469059	4142300.0	0.032835	0.032307	0.007211	
	...	...	...	...	...	...	...	...	
	768	2023-10-05	KO	50.808578	29878100.0	-0.048329	-0.049535	0.014405	
	861	2023-10-05	PEP	155.269150	10556400.0	-0.052158	-0.053567	0.001186	
	592	2023-10-05	CLX	119.689247	7595500.0	-0.052340	-0.053760	0.000560	
	797	2023-10-05	MDLZ	63.479565	7626600.0	-0.052563	-0.053994	-0.026631	
	925	2023-10-05	TAP	57.288155	3523000.0	-0.062986	-0.065057	0.002368	

497 rows × 8 columns

In [357...

```
# we can also look at the demeaned returns, or the returns of the asset we c
# this will help us assess if we were able to choose the assets that would e
```

```

date_mean_logreturns = hdf.groupby('Date')['logreturns'].transform('mean')

# Calculate the demeaned log returns
hdf['demeaned_logreturn'] = hdf['logreturns'] - date_mean_logreturns

# Similar to the above, calculate the mean *forward* log returns for each date
date_mean_fwd_logreturns = hdf.groupby('Date')['fwd_logreturn'].transform('mean')

# Calculate the demeaned forward log returns
hdf['demeaned_fwd_logreturn'] = hdf['fwd_logreturn'] - date_mean_fwd_logreturns
hdf.head()

```

Out [357...

	Price	Date	Ticker	Adj Close	Volume	returns	logreturns	fwd_logreturn	l
497		2023-10-05	A	109.606766	1295700.0	-0.011821	-0.011891	0.002625	
498		2023-10-05	AAL	12.850000	36272900.0	0.009427	0.009382	-0.007029	
499		2023-10-05	AAPL	173.828293	48527900.0	0.007198	0.007172	0.014643	
500		2023-10-05	ABBV	140.840302	3599500.0	-0.001625	-0.001626	0.005343	
501		2023-10-05	ABNB	124.989998	7630200.0	-0.018994	-0.019177	0.010901	

In [358...

```

# for ease of analyzing the effect, we will bucket our feature using a decile
# this will help us figure out how strong the effect is
# we do this using pd.qcut(x, q=quantile, labels=False, duplicates='drop'),
# [https://pandas.pydata.org/docs/reference/api/pandas.qcut.html]
# drop labels and duplicates
hdf['logreturn_decile'] = hdf.groupby('Date')['logreturns'].transform(
    lambda x: pd.qcut(x, q=10, labels=False, duplicates='drop'))
hdf.head()

```

Out [358...

	Price	Date	Ticker	Adj Close	Volume	returns	logreturns	fwd_logreturn	l
497		2023-10-05	A	109.606766	1295700.0	-0.011821	-0.011891	0.002625	
498		2023-10-05	AAL	12.850000	36272900.0	0.009427	0.009382	-0.007029	
499		2023-10-05	AAPL	173.828293	48527900.0	0.007198	0.007172	0.014643	
500		2023-10-05	ABBV	140.840302	3599500.0	-0.001625	-0.001626	0.005343	
501		2023-10-05	ABNB	124.989998	7630200.0	-0.018994	-0.019177	0.010901	

In [359...

```

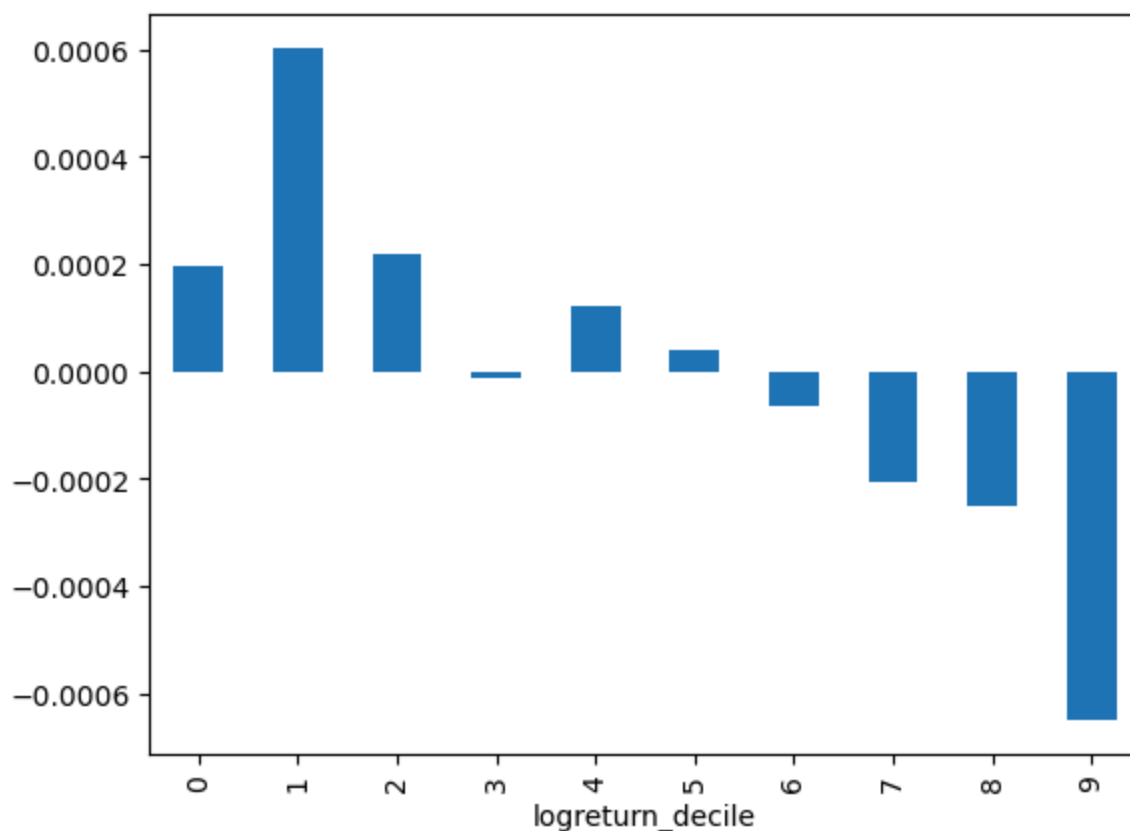
hdf.groupby('logreturn_decile').mean(numeric_only = True)['demeaned_fwd_logr

```

```
Out[359... logreturn_decile
0      0.000197
1      0.000603
2      0.000219
3     -0.000012
4      0.000123
5      0.000040
6     -0.000063
7     -0.000205
8     -0.000251
9     -0.000650
Name: demeaned_fwd_logreturn, dtype: float64
```

```
In [360... # we'll plot this using a barplot
hdf.groupby('logreturn_decile').mean(numeric_only = True)['demeaned_fwd_logr
```

```
Out[360... <Axes: xlabel='logreturn_decile'>
```



```
In [361... # it definitely looks like the bottom and top 20%-ile have outsized returns!
```

Strategy 'backtesting'

Backtesting: A Simplified Approach

While backtesting is a valuable tool, it can sometimes be excessive when conducting an **initial test** to evaluate whether a feature is useful. Instead, we'll start with a **basic backtest** using the following assumptions:

1. No Trading Costs:

- We'll ignore trading costs for now.
- Although trading costs are a significant drag on profitability, they're not critical for this initial test.

2. Key Idea:

- If a strategy **doesn't work** without trading costs, it's unlikely to work **with** trading costs included.

This simplified backtest provides a quick way to assess the viability of features before diving into more complex analyses.

```
In [362... # let's define our strategy by setting an asset to be long or short if its i
# we'll also equal weight all our positions
# the decile values for the bottom 20% would be 0 and 1
# the decile values for the top 20% would be 8 and 9

# we can use np.where() in order to conditionally assign a column
def longorshort(d):
    if d in [0, 1]:
        return 1
    elif d in [8, 9]:
        return -1
    else:
        return 0
hdf['long/short'] = hdf["logreturn_decile"].apply(longorshort)
hdf.head()
```

```
Out[362... Price    Date    Ticker    Adj Close    Volume    returns    logreturns    fwd_logreturn    I
```

497	2023-10-05	A	109.606766	1295700.0	-0.011821	-0.011891	0.002625	
498	2023-10-05	AAL	12.850000	36272900.0	0.009427	0.009382	-0.007029	
499	2023-10-05	AAPL	173.828293	48527900.0	0.007198	0.007172	0.014643	
500	2023-10-05	ABBV	140.840302	3599500.0	-0.001625	-0.001626	0.005343	
501	2023-10-05	ABNB	124.989998	7630200.0	-0.018994	-0.019177	0.010901	

```
In [363... # we size our positions by taking our absolute long/short position
hdf['absposition'] = np.abs(hdf["long/short"])
```

```
hdf.head()
```

```
Out[363... Price    Date    Ticker    Adj Close    Volume    returns    logreturns    fwd_logreturn    l
```

497	2023-10-05	A	109.606766	1295700.0	-0.011821	-0.011891	0.002625	
498	2023-10-05	AAL	12.850000	36272900.0	0.009427	0.009382	-0.007029	
499	2023-10-05	AAPL	173.828293	48527900.0	0.007198	0.007172	0.014643	
500	2023-10-05	ABBV	140.840302	3599500.0	-0.001625	-0.001626	0.005343	
501	2023-10-05	ABNB	124.989998	7630200.0	-0.018994	-0.019177	0.010901	

```
In [364... # getting the total number of our positions by summing our absolute position  
# hint: take a look at .transform documentation [https://pandas.pydata.org/c  
hdf['numpositions'] = hdf.groupby('Date')['absposition'].transform('sum')  
hdf.head()
```

```
Out[364... Price    Date    Ticker    Adj Close    Volume    returns    logreturns    fwd_logreturn    l
```

497	2023-10-05	A	109.606766	1295700.0	-0.011821	-0.011891	0.002625	
498	2023-10-05	AAL	12.850000	36272900.0	0.009427	0.009382	-0.007029	
499	2023-10-05	AAPL	173.828293	48527900.0	0.007198	0.007172	0.014643	
500	2023-10-05	ABBV	140.840302	3599500.0	-0.001625	-0.001626	0.005343	
501	2023-10-05	ABNB	124.989998	7630200.0	-0.018994	-0.019177	0.010901	

```
In [365... # and we get our final weight for each asset by scaling our long/short indic  
# each position should be such that we add up to one, so we'd divide the inc  
# if we have 0 positions, the weight should be 0  
# this will mean that we should have equal size long and short, adding up to  
# we'll use df.apply(lambda row: (expression using row) if (condition on son  
  
hdf['weight'] = hdf['long/short'] / hdf['numpositions']  
hdf.head()
```

Out [365...

Price	Date	Ticker	Adj Close	Volume	returns	logreturns	fwd_logreturn	I
497	2023-10-05	A	109.606766	1295700.0	-0.011821	-0.011891	0.002625	
498	2023-10-05	AAL	12.850000	36272900.0	0.009427	0.009382	-0.007029	
499	2023-10-05	AAPL	173.828293	48527900.0	0.007198	0.007172	0.014643	
500	2023-10-05	ABBV	140.840302	3599500.0	-0.001625	-0.001626	0.005343	
501	2023-10-05	ABNB	124.989998	7630200.0	-0.018994	-0.019177	0.010901	

```
In [366... # finally we define the strategy's return as the weighted logreturns based c
# so we multiply weight by forward logreturn
hdf['strategy_logreturn'] = hdf['weight'] * hdf['fwd_logreturn']
hdf.head()
```

Out [366...	Price	Date	Ticker	Adj Close	Volume	returns	logreturns	fwd_logreturn	I
	497	2023-10-05	A	109.606766	1295700.0	-0.011821	-0.011891	0.002625	
	498	2023-10-05	AAL	12.850000	36272900.0	0.009427	0.009382	-0.007029	
	499	2023-10-05	AAPL	173.828293	48527900.0	0.007198	0.007172	0.014643	
	500	2023-10-05	ABBV	140.840302	3599500.0	-0.001625	-0.001626	0.005343	
	501	2023-10-05	ABNB	124.989998	7630200.0	-0.018994	-0.019177	0.010901	

```
In [367... # we can compare this with an equal weighted strategy, where we long each as
# hint: use .unique() to get number of assets [https://pandas.pydata.org/doc

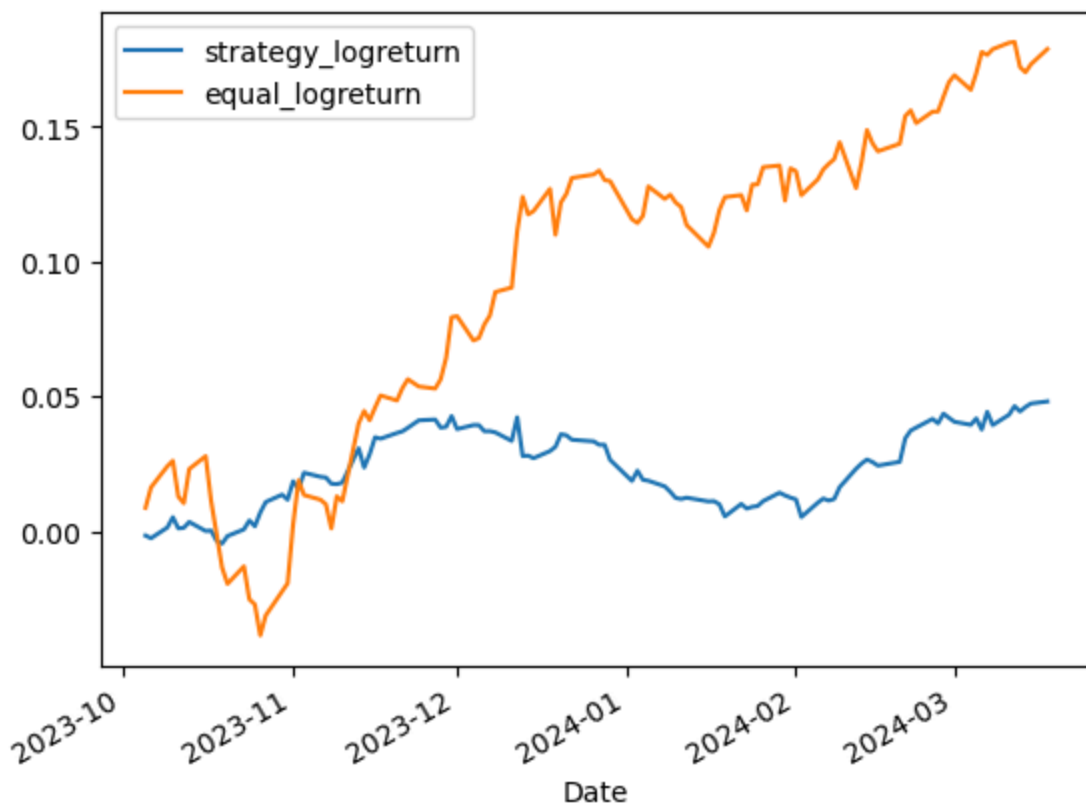
hdf['eqweight'] = 1 / len(pd.unique(hdf['Ticker']))
hdf['equal_logreturn'] = hdf['eqweight'] * hdf['fwd_logreturn']
hdf.head()
```


	Price	Date	Ticker	Adj Close	Volume	returns	logreturns	fwd_logreturn	I
497		2023-10-05	A	109.606766	1295700.0	-0.011821	-0.011891	0.002625	
498		2023-10-05	AAL	12.850000	36272900.0	0.009427	0.009382	-0.007029	
499		2023-10-05	AAPL	173.828293	48527900.0	0.007198	0.007172	0.014643	
500		2023-10-05	ABBV	140.840302	3599500.0	-0.001625	-0.001626	0.005343	
501		2023-10-05	ABNB	124.989998	7630200.0	-0.018994	-0.019177	0.010901	

Evaluating performance

```
In [368... hdf.groupby('Date').sum()['strategy_logreturn'].cumsum().plot(legend=True)
hdf.groupby('Date').sum()['equal_logreturn'].cumsum().plot(legend=True)
```

```
Out[368... <Axes: xlabel='Date'>
```



```
In [369... # we can get the final return by getting the last value of the column (value
# and we can translate that to simple returns for readability
final_strategyreturn = logtosimple(hdf.groupby('Date').sum()['strategy_logre
final_equalreturn = logtosimple(hdf.groupby('Date').sum()['equal_logreturn']
```

```
print(f'final_strategyreturn: {final_strategyreturn}')
print(f'final_equalreturn: {final_equalreturn}')
```

```
final_strategyreturn: 0.04924253346870078
final_equalreturn: 0.19573618511585034
```

Evaluating Strategy Performance

The equal-weight strategy significantly outperformed our strategy. Given the simplicity of our approach, this outcome is not unexpected. However, the equal-weight strategy also exhibited notable **volatility**.

To better account for volatility, we can use the **Sharpe Ratio**.

Understanding the Sharpe Ratio

The Sharpe Ratio measures the **risk-adjusted return** of a strategy and can be thought of as a t-test for the statistical significance of the returns. It is calculated as:

$$\text{Sharpe Ratio} = \frac{\text{Mean Return} - \text{Risk-Free Rate}}{\text{Standard Deviation of Returns}}$$

Generally, any sharpe ratio over 1 is good, 2 is very good, 3+ is very very good. As you see a sharpe ratio > 3, the more likely that the strategy is somehow limited, or you've calculated something wrong

Important Caveats

In this example, the calculated Sharpe Ratios are **not realistic** due to several simplifications. For instance:

- **Lookahead Bias:** Restricting the universe to companies currently in the S&P 500 introduces bias, as we are using information unavailable in past periods (e.g., in 2023).

There are likely other ways this sample or testing strategy might be limited. Consider exploring additional potential biases or flaws in the approach!

```
In [370]: # let's calculate the sharpe ratio with a function, we'll leave out the risk
# we also need to normalize the sharpe ratio with respect to a year, by mult
# note that there are 252 trading days

def sharpe_ratio(mean_ret, std_ret):
    return (mean_ret / std_ret) * np.sqrt(252)

strat_sharpe = sharpe_ratio(hdf.groupby('Date').sum()['strategy_logreturn'].
                           hdf.groupby('Date').sum()['strategy_logreturn'].std())
equal_sharpe = sharpe_ratio(hdf.groupby('Date').sum()['equal_logreturn'].mea
```

```
hdf.groupby('Date').sum()['equal_logreturn'].std())
print(strat_sharpe, equal_sharpe)
```

1.840395868545438 3.1044873685151626

My Strategy: Fourier analysis on log returns

```
In [372]: # take a single asset
# take the list of log returns associated with the asset
aapl = hdf.groupby("Ticker").get_group("EQR")["logreturns"]
#aapl = aapl[0:len(aapl)//2]
aapl = aapl.to_numpy()
# plot it

# take the first half and perform fft
def perform_fft(sequence):
    # Perform the Fast Fourier Transform
    fft_result = np.fft.fft(sequence)
    # Compute the corresponding frequencies
    n = len(sequence)
    freq = np.fft.fftfreq(n)
    return freq, fft_result
def recover_signal(fft_result):
    # Perform the Inverse Fast Fourier Transform to recover the original signal
    recovered_signal = np.fft.ifft(fft_result)
    return np.real(recovered_signal) # Return only the real part (original)
def extrapolate_signal(fft_result, num_points):
    # Extend the FFT result by zero-padding to extrapolate the time series
    n = len(fft_result)
    extended_fft = np.zeros(num_points, dtype=complex)
    extended_fft[:n] = fft_result

    # Perform the inverse FFT on the extended data
    extrapolated_signal = np.fft.ifft(extended_fft)
    return np.real(extrapolated_signal) # Return only the real part

"""
window_len = 24
freq, fft_result = perform_fft(aapl[:window_len])
fft_result[7:len(aapl)-7] = 0
#print(fft_result)
#print(freq)
aapl_fourier = recover_signal(fft_result)

plt.plot(range(0, len(aapl_fourier))[:window_len], aapl[:window_len])
print(len(aapl))
plt.plot(range(0, len(aapl_fourier))[:window_len], aapl_fourier[:window_len])
plt.legend(loc="upper right")
plt.show()

plt.plot(range(0, len(aapl)), aapl)
print(len(aapl) // window_len)
#print(np.concatenate(2 * (aapl_fourier,)))
plt.plot(range(0, len(aapl)), np.concatenate((1 + len(aapl)//window_len)*(aapl
```

```
plt.legend(loc="upper right")
plt.show()
"""

"""
plt.plot(freq[:len(freq)//2], np.abs(fft_result)[:len(freq)//2])
plt.title("FFT Result (Magnitude)")
plt.xlabel("Frequency")
plt.ylabel("Magnitude")
plt.show()
"""

# plot the waveform from the fft onto the full data and see if there is any
```

```
Out[372... '\nplt.plot(freq[:len(freq)//2], np.abs(fft_result)[:len(freq)//2])\nplt.ti
tle("FFT Result (Magnitude)")\nplt.xlabel("Frequency")\nplt.ylabel("Magnitu
de")\nplt.show()\n'
```

```
In [373... # eliminate high frequency components with wavelength <= 3
"""

cutoff_wavelength = 5
cutoff_freq = 1/cutoff_wavelength
cutoff_n = 1 // cutoff_wavelength
fft_result[:cutoff_n:] = 0
fft_result[l-cutoff_n:] = 0
"""

def predict_fwd_logreturn(windowed_series):
    window_lens = range(12, 6, -1)
    test_len = 12
    mses = []
    ls = []
    signals = []
    for l in window_lens:
        # perform fft on a window of length l, excluding 5 data points after
        to_fit = windowed_series[-l-test_len:-test_len]
        freq, fft_result = perform_fft(to_fit)

        # remove insignificant frequency components
        #max_amp = np.max(np.abs(fft_result))
        #fft_result[np.abs(fft_result) < max_amp * 0.625] = 0

        # reconstruct original signal
        lowpass_signal = recover_signal(fft_result)

        extended_signal = np.concatenate(((l+test_len+1+ l) // l)*(lowpass_s
        #print(l)
        #print(len(extended_signal))

        # measure how close the last 5 data points match up with the first 5
        mse = np.mean(np.square(extended_signal[-test_len-1:-1] - windowed_s
        mses.append(mse)
        ls.append(l)
        signals.append(extended_signal)

    # find effective frequency of data points
    min_error = 1e9
```

```

min_i = -1
for i in range(len(mses)):
    if mses[i] < min_error:
        min_error = mses[i]
        min_i = i

#print("Freq: ", ls[min_i])
#plt.plot(range(ls[min_i]+test_len), windowed_series[-ls[min_i]-test_len:ls[min_i]+test_len])
#plt.plot(range(ls[min_i]+test_len), signals[min_i][:-1])
#plt.legend(loc="upper right")
#plt.show()

return signals[min_i][:-1]

# assumed for one asset
def calculate_positions(single_asset):
    #g = data.groupby('Ticker')
    #tickers = g.groups.keys()
    #print(tickers)
    #grp_name = "AAPL"
    #single_asset = g.get_group(grp_name)

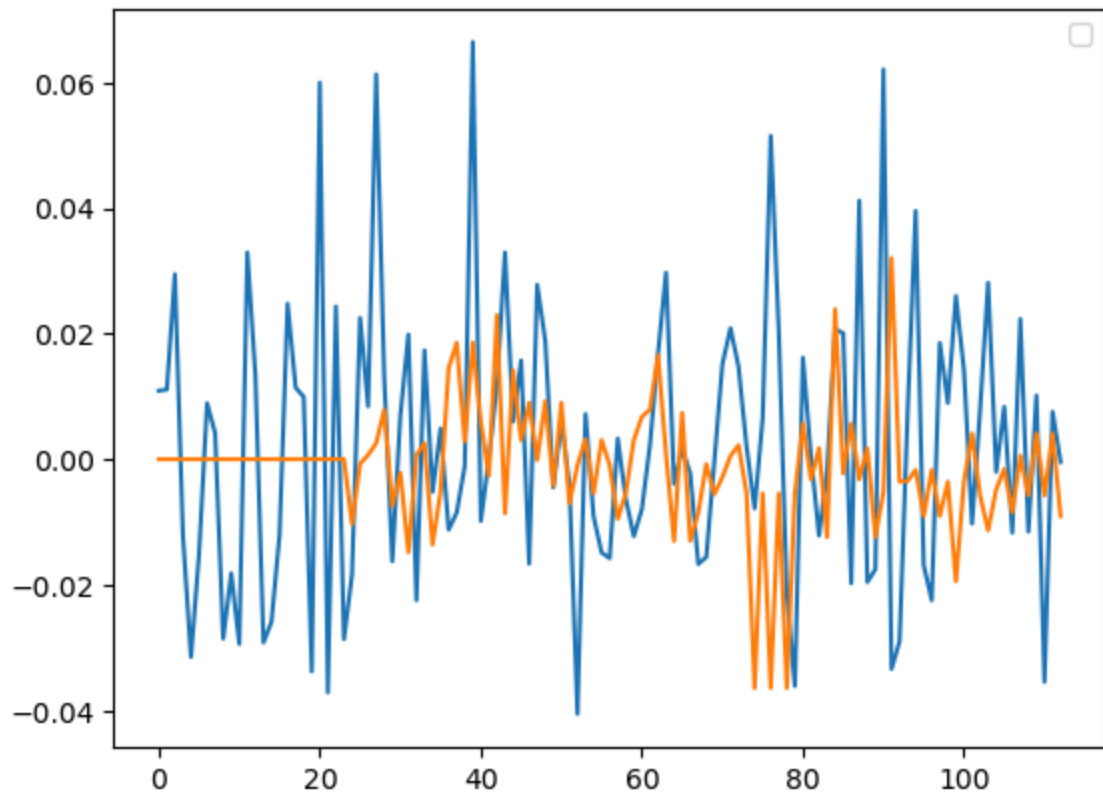
    fwds_list = np.zeros(single_asset.shape[0])
    window_len = 24
    for i in range(window_len, len(fwds_list)):
        #print(f"Day {i}")
        window_logreturns = single_asset[i-window_len:i].to_numpy()
        #pstn = calculate_single_position(window_logreturns)
        fwd = predict_fwd_logreturn(window_logreturns)
        fwds_list[i] = fwd
    #print(positions)
    return fwds_list

asset_fwdlogreturns = hdf.groupby("Ticker").get_group("ABNB")["fwd_logreturns"]
asset_fwd = calculate_positions(hdf.groupby("Ticker").get_group("AAPL")["logreturns"])

plt.plot(range(0, len(asset_fwdlogreturns)), asset_fwdlogreturns)
plt.plot(range(0, len(asset_fwdlogreturns)), asset_fwd)
plt.legend(loc="upper right")
plt.show()

#hdf["newstrat_longshort"] = hdf.groupby("Ticker")["logreturns"].transform(c

```



```
In [374...] hdf["newstrat_predictedlogret"] = hdf.groupby("Ticker")["logreturns"].transf
```

```
In [375...] hdf.tail(10)
```

Out [375...

Price	Date	Ticker	Adj Close	Volume	returns	logreturns	fwd_logreturn
56648	2024-03-18	WTW	271.214355	402800.0	0.006876	0.006853	-0.003219
56649	2024-03-18	WY	33.697689	3373100.0	-0.005243	-0.005257	0.011066
56650	2024-03-18	WYNN	97.998962	1528700.0	-0.002523	-0.002526	0.009165
56651	2024-03-18	XEL	51.073341	7217700.0	0.000962	0.000961	0.007466
56652	2024-03-18	XOM	109.604904	16770500.0	0.009257	0.009214	0.007010
56653	2024-03-18	XYL	126.820251	720000.0	0.001729	0.001727	0.007502
56654	2024-03-18	YUM	134.930374	2384500.0	-0.004165	-0.004174	0.007457
56655	2024-03-18	ZBH	125.166573	972300.0	0.006312	0.006292	0.005542
56656	2024-03-18	ZBRA	285.970001	329300.0	0.000385	0.000385	-0.002731
56657	2024-03-18	ZTS	169.399384	4955000.0	-0.010894	-0.010954	0.013211

In [376...

```
# give weights to newstrat weights
hdf["newstrat_abspos"] = hdf.groupby("Date")["newstrat_predictedlogret"].tra
hdf["newstrat_totalpos"] = hdf.groupby("Date")["newstrat_abspos"].transform(
hdf["newstrat_weight"] = hdf["newstrat_predictedlogret"] / hdf["newstrat_tot
```

In [377...

```
hdf.tail()
```

Out [377...

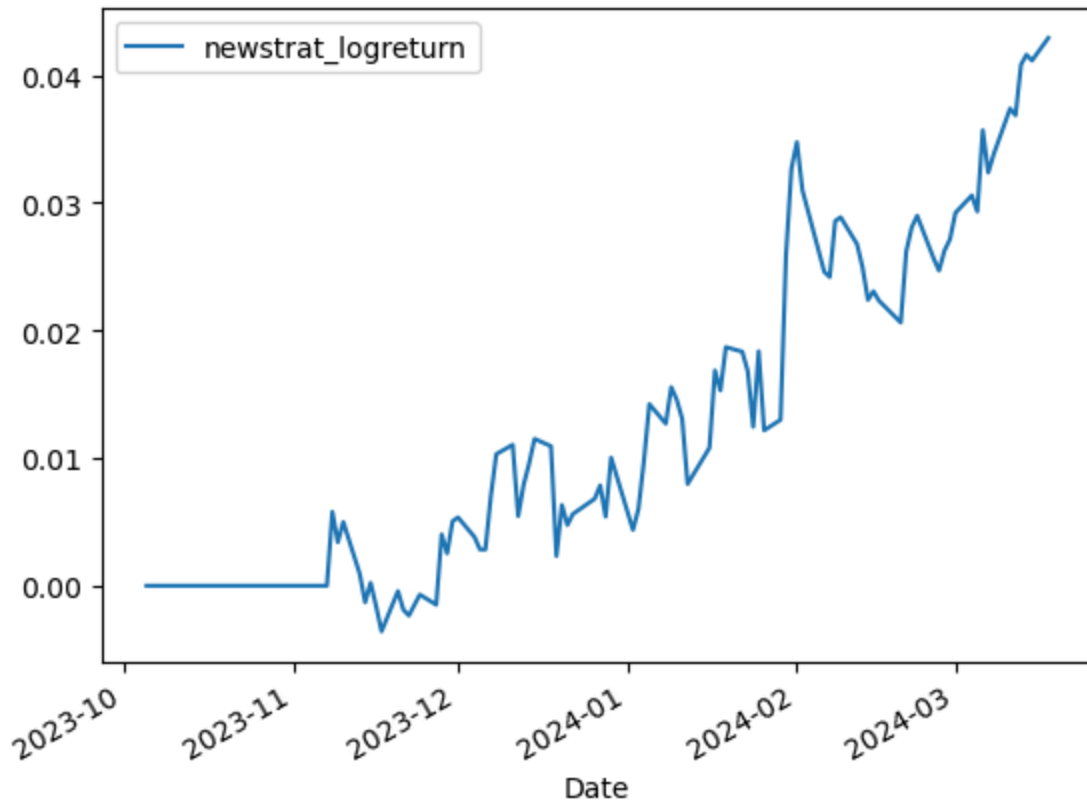
Price	Date	Ticker	Adj Close	Volume	returns	logreturns	fwd_logreturn
56653	2024-03-18	XYL	126.820251	720000.0	0.001729	0.001727	0.007502
56654	2024-03-18	YUM	134.930374	2384500.0	-0.004165	-0.004174	0.007457
56655	2024-03-18	ZBH	125.166573	972300.0	0.006312	0.006292	0.005542
56656	2024-03-18	ZBRA	285.970001	329300.0	0.000385	0.000385	-0.002731
56657	2024-03-18	ZTS	169.399384	4955000.0	-0.010894	-0.010954	0.013211

5 rows × 22 columns

```
In [378... # calculate return of this strategy
hdf['newstrat_logreturn'] = hdf['newstrat_weight'] * hdf['fwd_logreturn']
```

```
In [379... #hdf.groupby('Date').sum()['strategy_logreturn'].cumsum().plot(legend=True)
#hdf.groupby('Date').sum()['equal_logreturn'].cumsum().plot(legend=True)
hdf.groupby('Date').sum()['newstrat_logreturn'].cumsum().plot(legend=True)
```

Out[379... <Axes: xlabel='Date'>



```
In [380... hdf.groupby('Date').sum()['strategy_logreturn'].cumsum().plot(legend=True)
hdf.groupby('Date').sum()['equal_logreturn'].cumsum().plot(legend=True)
hdf.groupby('Date').sum()['newstrat_logreturn'].cumsum().plot(legend=True)
```

Out[380... <Axes: xlabel='Date'>

